HTTP methods and status codes

HTTP methods and status codes form the foundational contract between client-side applications (frontend, mobile, external systems) and your server architecture. Correctly implementing these standards ensures your APIs are predictable, secure, and compliant with caching and infrastructure protocols.

1. Core HTTP Methods (Verbs)

HTTP methods map directly to actions taken on resources, roughly correlating with database CRUD (Create, Read, Update, Delete) operations. Two critical constraints define how backend engineers choose their methods:

Safe Methods: Do not modify the server's database state or cause side effects (e.g., executing a calculation or fetching a record).

Idempotent Methods: Sending the exact same request multiple times leaves the system in identical final states.

HTTP Method	Core Action	Idempotent	Safe	Typical DB Equivalent	Backend Design Nuance
GET	Read	**Yes**	**Yes**	`SELECT`	Must never mutate data. Edge proxies and CDNs automatically cache successful `GET` responses.
POST	Create	**No**	**No**	`INSERT`	Not idempotent. Multiple identical submissions create multiple distinct database entries.
PUT	Replace / Upsert	**Yes**	**No**	`UPDATE` (Full record)	Expects a *complete payload* of the resource. Missing fields are typically wiped out or set to default values.
PATCH	Partial Update	**No**	**No**	`UPDATE` (Specific column)	Sends only the key-value pairs that need to change. Highly

HTTP Method	Core Action	Idempotent	Safe	Typical DB Equivalent	Backend Design Nuance
					useful for minimizing network payload weight.
DELETE	Remove	**Yes**	**No**	`DELETE`	Removes the target resource. Deleting a resource multiple times returns the same final data structure (empty/removed).
OPTIONS	Pre-flight check	**Yes**	**Yes**	*N/A*	Used dynamically by modern web browsers to verify server security constraints (like CORS policies) before sending cross-origin mutations.

2. HTTP Status Code Architecture

A server communicates the success, migration, failure, or security block of a request using three-digit numbers categorized into five distinct blocks.

🟢 2xx: Success (The request worked)

`200 OK`: Standard success indicator. Commonly returned alongside `GET` payloads or successful state-changing operations where the updated data is returned.

`201 Created`: Returned for a successful `POST` request. Best practice dictates returning the newly generated resource ID or a `Location` header directing the client to the new asset.

`202 Accepted`: The request is valid and has been queued for background processing (e.g., microservice pub/sub event, video encoding job).

`204 No Content`: The operation succeeded, but there is intentionally no payload payload to return (extremely common for `DELETE` tasks).

🟡 3xx: Redirection (Go somewhere else)

`301 Moved Permanently`: The resource resides at a new permanent URI. Browsers cache this redirection automatically.

`304 Not Modified`: Tells the client's device to read from its local application cache instead of asking the database to transfer the payload again (highly optimized for asset delivery).

🔵 4xx: Client Errors (The caller made a mistake)

`400 Bad Request`: The client sent malformed payloads, missed mandatory keys, or broke structural application validation rules.

`401 Unauthorized`: Missing or invalid session cookies, JWT tokens, or authentication credentials. **The user cannot be authenticated.**

`403 Forbidden`: The server accurately knows who the user is, but their permission level or ACL role does not allow this action. **Authenticated, but lacking access rights.**

`404 Not Found`: The unique identifier or route requested does not correspond to an existing record.

`409 Conflict`: State conflict on the database layer (e.g., trying to create an account with an email address that is already registered).

`422 Unprocessable Entity`: Formally accurate syntax, but semantic logical errors (e.g., fields containing impossible data configurations).

`429 Too Many Requests`: The client has breached rate-limit boundaries. Often returned with a `Retry-After` header telling the client when it's safe to resume.

🔴 5xx: Server Errors (Your system broke)

`500 Internal Server Error`: An unhandled exception or critical crash occurred somewhere within your application code.

`502 Bad Gateway`: The reverse-proxy server (like Nginx, AWS ALB, Cloudflare) could not receive a valid or timely response from your running app engine container.

`503 Service Unavailable`: The server infrastructure is physically overwhelmed, undergoing immediate database migrations, or completely unresponsive.

`504 Gateway Timeout`: An internal database query or microservice API call took longer to execute than your edge routing gateway allowed.

---

3. Essential Architectural Anti-Patterns to Avoid

The "Everything is 200 OK" Syndrome: Returning a `200 OK` status with a body containing `{ "error": "User not found" }` ruins network caching layers, makes security firewalls blind to attacks, and complicates client-side parsing pipelines.

Leaking Database Stack Traces on `500`: Never print actual system syntax exceptions or database error blocks inside raw production response schemas. Malicious actors use exposed stack traces to pinpoint framework vulnerabilities.

Using `GET` for Mutating Operations: Crafting paths like `GET /api/users/delete/45` can trigger unintentional deletions if an automated web-crawler follows the URL string.

If you are currently mapping out a project, let me know:

What **backend language or framework** are you using? (e.g., Node/Express, Go, FastAPI, Spring Boot)

What **specific API scenario** are you designing? (e.g., a file upload system, an asynchronous payment route, a user registration pipeline)

